

SubTrack

Design & Documentation Artefacts

Course	Python Essentials (Build Your Own Project)
Institution	VIT Bhopal University
Program	B.Tech Computer Science and Engineering (Core)
Student Name	Enayet Ali
Registration Number	26BCE10256
GitHub Repository	https://github.com/laptopenayet24-ui/subtrack-subscription-manager
Submission Date	30-09-2026

This document collects the design artefacts required by Section 4 of the VITyarthi *Build Your Own Project* guidelines: problem statement, objectives, functional and non-functional requirements, system architecture, workflow, UML diagrams and the storage design.

1. Problem Statement

Most people sign up for several recurring subscriptions, such as streaming, cloud storage, online learning and software tools, and lose track of them over time. Without a single place to see everything, it is easy to miss a renewal date, or to keep paying for a service that is no longer used.

Scope. SubTrack is a command-line Python program that lets a user manage subscriptions in one place: adding, viewing and deleting them, and checking the total monthly and yearly cost. All data is stored locally in a JSON file, so no database server or internet connection is needed.

Target users.

- Individuals who want a simple way to track their monthly and yearly subscription costs.
- Budget-conscious users who want a clearer picture of their recurring expenses.

2. Objectives

- Build a Python console application that lets a user add, view and delete subscription entries without a database server or external dependencies.
- Give the user a financial overview that correctly combines monthly and yearly billing cycles.
- Store subscription data reliably in a local JSON file so that every successful add or delete is still there the next time the program runs.
- Keep the code modular and maintainable by separating the presentation layer (`main.py`) from the business logic layer (`services/subscription_manager.py`).
- Keep the solution lightweight and dependency-free, so that it runs anywhere Python 3 is installed.

3. Functional Requirements

SubTrack is organised into three major functional modules, each with a clear input/output structure. The user drives everything from a numbered text menu.

Module 1 – Subscription Management (CRUD)

	Details
Inputs	Menu choice 1, 2 or 3; for Add: name, cost, billing cycle (monthly/yearly), category, renewal date (DD-MM-YYYY); for Delete: subscription name.
Processing	Create a new subscription entry; list the current user's entries; remove the named entry.
Outputs	Formatted list of subscriptions, or a confirmation / error message.

Module 2 – Financial Summary & Reporting

	Details
Inputs	Menu choice 4.
Processing	Total the user's subscription costs. For the yearly figure, monthly costs are multiplied by 12 and yearly costs are added as they are.
Outputs	Total monthly spend and total yearly spend.

Module 3 – User Session & Data Persistence

	Details
Inputs	Username entered at start-up.
Processing	initialize_app loads the user's record from subscriptions_data.json or creates a new profile; every add or delete is saved back to the file.
Outputs	The user's profile ready for use; the JSON file kept up to date.

Function-level input/output

Function	Called from	Responsibility
initialize_app(username)	main(), at start-up	Loads the user's record from the JSON store, or creates a new one if the username is not present yet.
view_subscriptions(user)	Menu options 1 and 3	Prints every subscription stored for the active user.
add_new_subscription(user, name, cost, cycle, category, renewal_date)	Menu option 2	Adds a new subscription entry to the user's record and saves the data.
delete_subscription(user, name)	Menu option 3	Removes the named subscription from the user's record and saves the change.
calculate_total_expenses(user)	Menu option 4	Works out the monthly and yearly totals across the user's subscriptions and prints them.

4. Non-Functional Requirements

Requirement	How SubTrack addresses it
Performance	Every operation works on one user's small in-memory list, so menu actions complete instantly. No network calls or heavy computation are involved.
Usability	A numbered menu with plain-English prompts (for example “Enter cost”). The full menu is shown again after every action.
Reliability	Data is written to the JSON file after every add or delete, so a confirmed change is not lost if the program is closed.
Maintainability	Presentation code (main.py) and business logic (services/subscription_manager.py) are in separate files, so each can be read, tested and extended on its own.
Error handling	The cost is parsed inside a try/except block. A non-numeric value produces a clear message and returns to the menu instead of crashing; an invalid menu choice does the same.
Resource efficiency	Only the built-in json and os modules are used, so there is nothing to install and memory use stays minimal.

5. System Architecture Diagram

SubTrack uses a three-layer architecture that separates how the user interacts with the program from how the data is processed and stored.

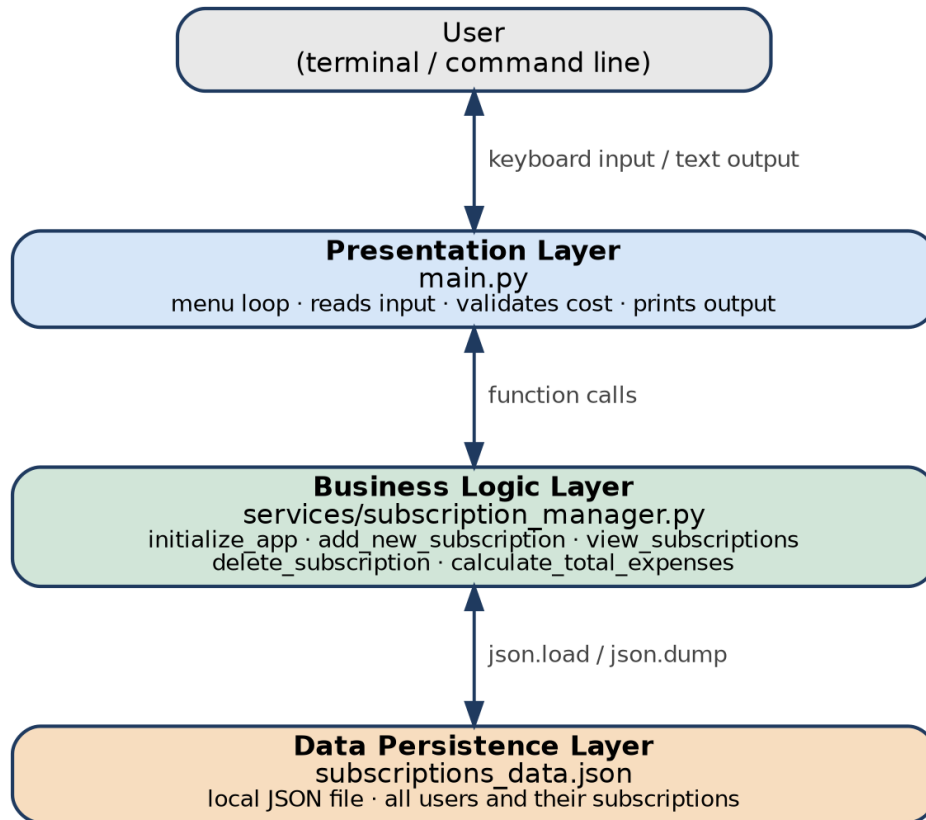


Figure 1: Layered system architecture of SubTrack.

6. Process Flow / Workflow Diagram

The workflow runs from start-up (username and profile loading) through the menu loop until the user chooses Exit. Add and Delete save to the JSON file before control returns to the menu.

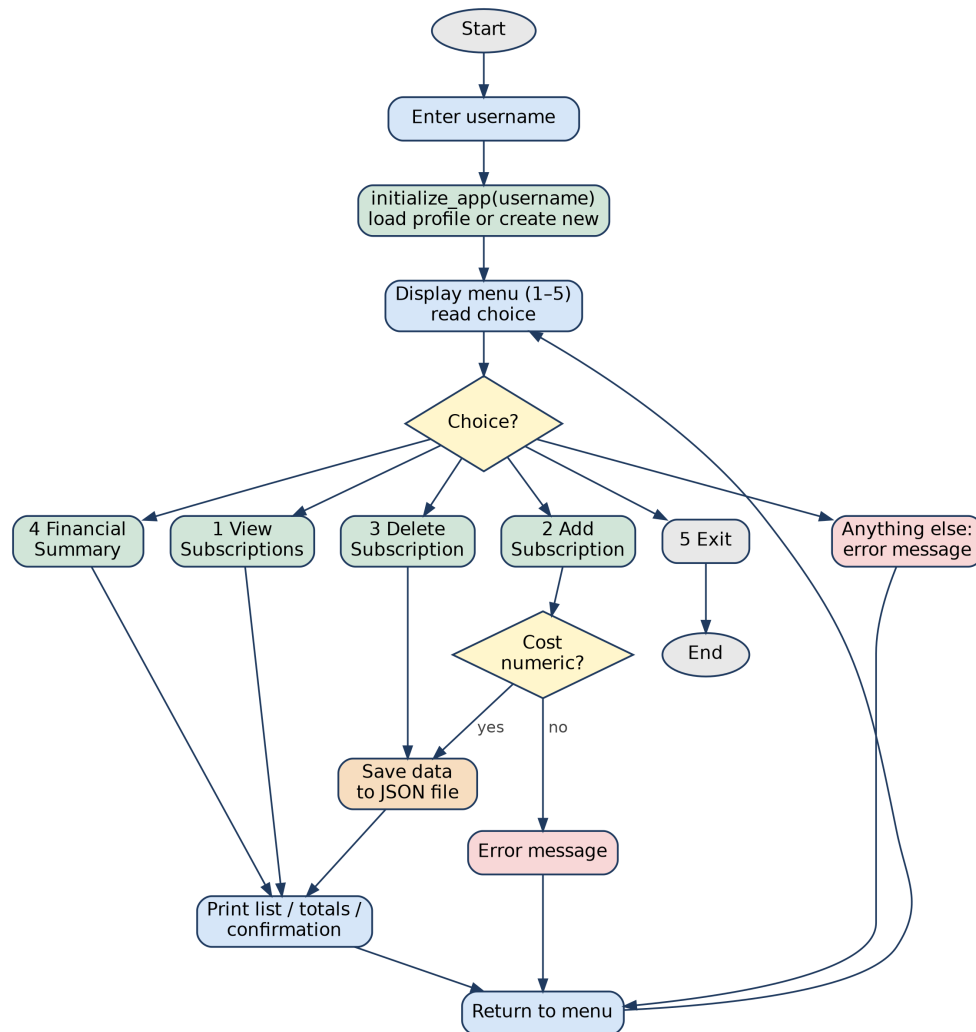


Figure 2: End-to-end workflow of SubTrack.

7. UML Diagrams

7.1 Use Case Diagram

SubTrack has one actor, the User. Adding or deleting a subscription includes saving the data to the JSON file.

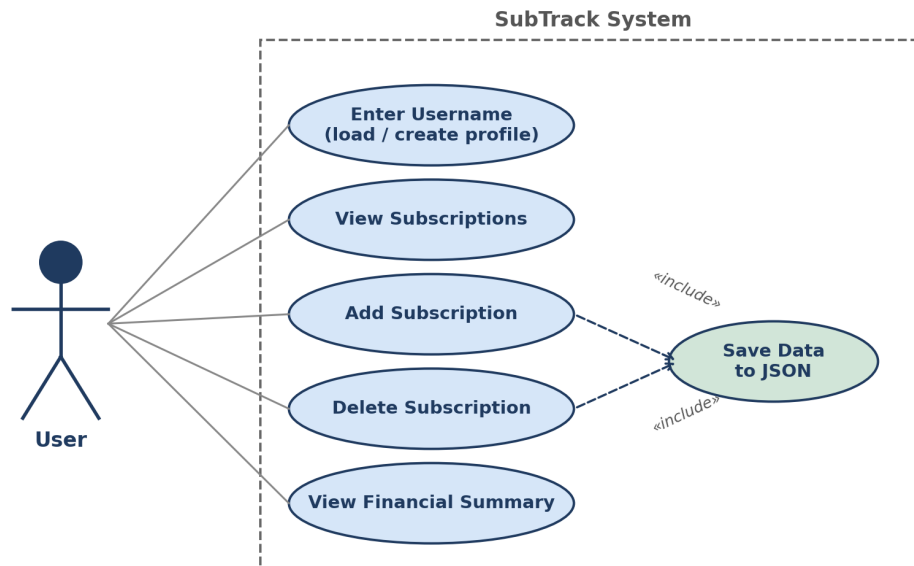


Figure 3: Use cases of SubTrack.

7.2 Component Diagram

SubTrack is implemented with functions rather than classes, so a component diagram is used instead of a class diagram. It shows how main.py, subscription_manager.py and the JSON file depend on each other.

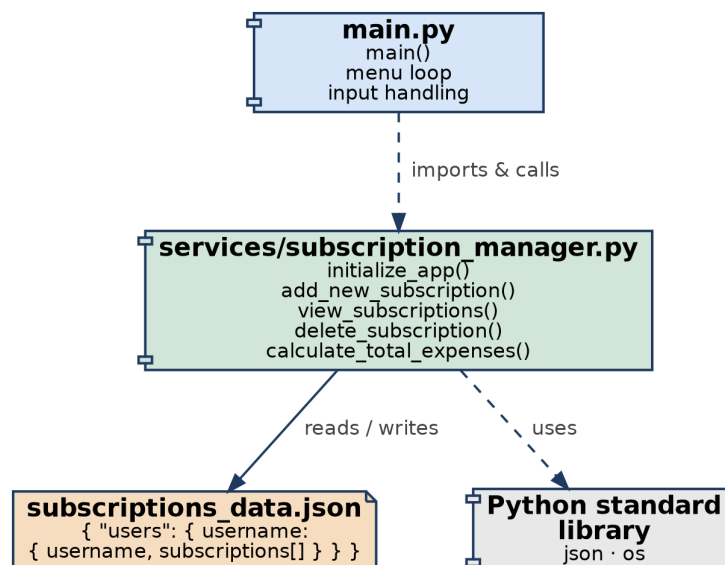


Figure 4: Component diagram of SubTrack.

7.3 Sequence Diagram – Add Subscription

Message flow when the user adds a subscription, including the cost check in `main.py`.

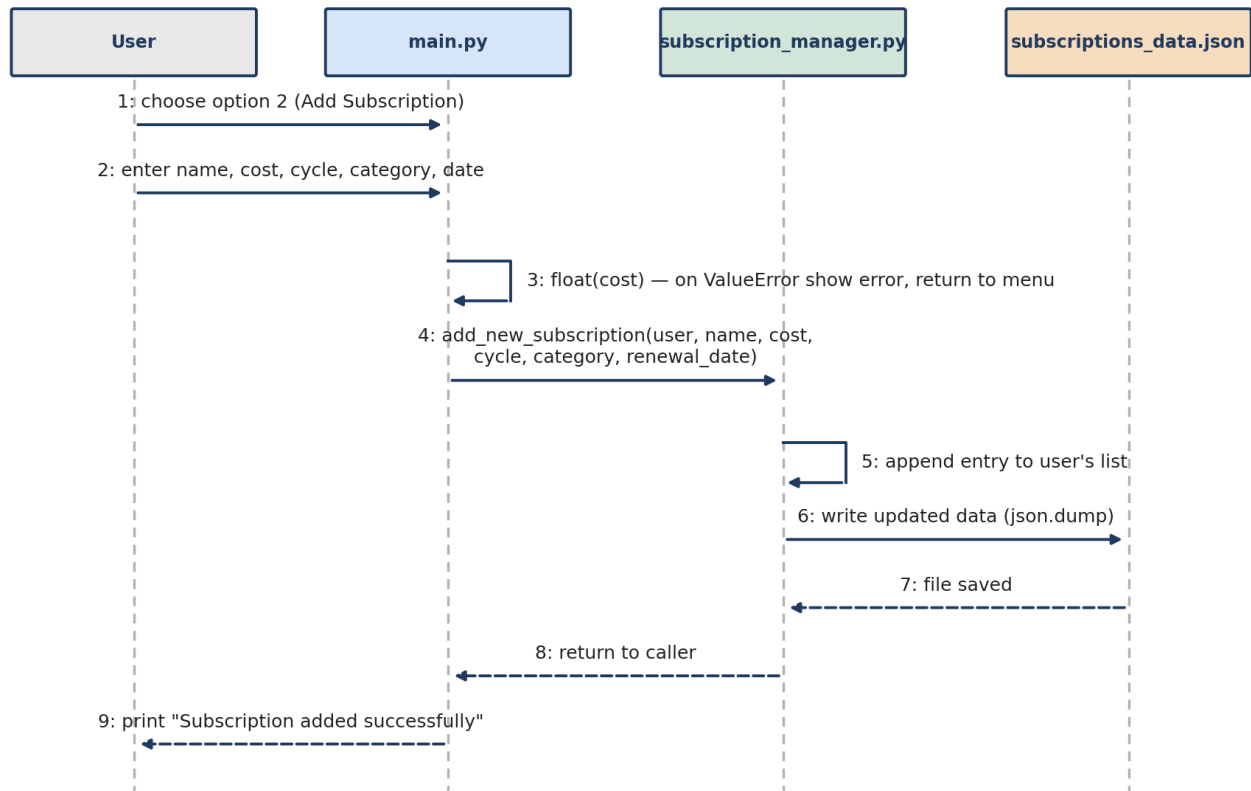


Figure 5: Sequence diagram for adding a subscription.

8. Database / Storage Design

8.1 ER Diagram

SubTrack stores its data in a JSON file. The logical entities are USER and SUBSCRIPTION: one user owns zero or more subscriptions. In the JSON file this relationship is expressed by nesting, so the SUBSCRIPTION entity's link to its owner is implicit; the username foreign key is shown in the diagram to make the relationship explicit. A subscription is identified within a user by its name.

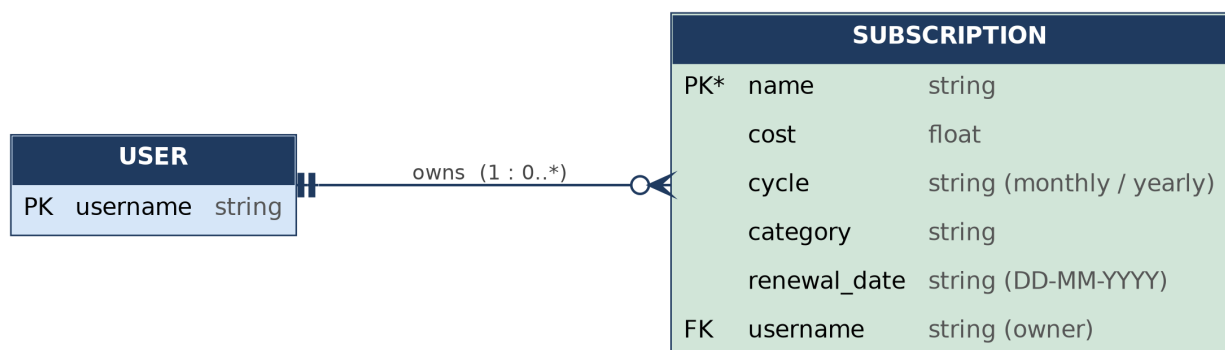


Figure 6: Entity-relationship diagram (USER 1 : 0..* SUBSCRIPTION).

8.2 Schema Design

The file `subscriptions_data.json` holds one top-level object, `users`, keyed by `username`. Each user object contains a `subscriptions` list.

Field	Type	Notes
username	string	Unique key that identifies the user record
subscriptions	list	Zero or more subscription objects
name	string	Service name, for example “Netflix”. Used to pick the entry to delete
cost	float	Amount billed, in the user's local currency (Rs.)
cycle	string	“monthly” or “yearly”
category	string	For example “Entertainment” or “Work”
renewal_date	string	Next renewal date in DD-MM-YYYY format

Example (taken from the structure of the sample data file):

```
{
  "users": {
    "Enayet Ali": {
      "username": "Enayet Ali",
      "subscriptions": [
        { "name": "Netflix", "cost": 299.0, "cycle": "monthly",
          "category": "Entertainment", "renewal_date": "29-09-2026" },
        { "name": "coursera", "cost": 6000.0, "cycle": "yearly",
          "category": "Work", "renewal_date": "31-12-2026" }
      ]
    }
  }
}
```